Part-3: Introduction to the tutorial

Why agentic systems for science?

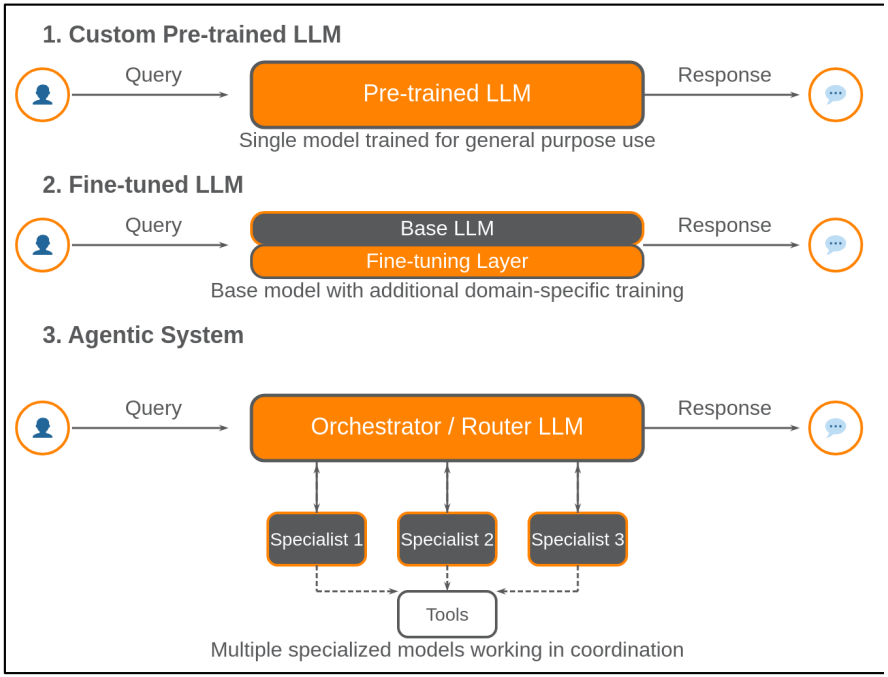
Custom Agentic systems vs commercial LLM interfaces

Limitations in LLMs can be overcome with agentic systems

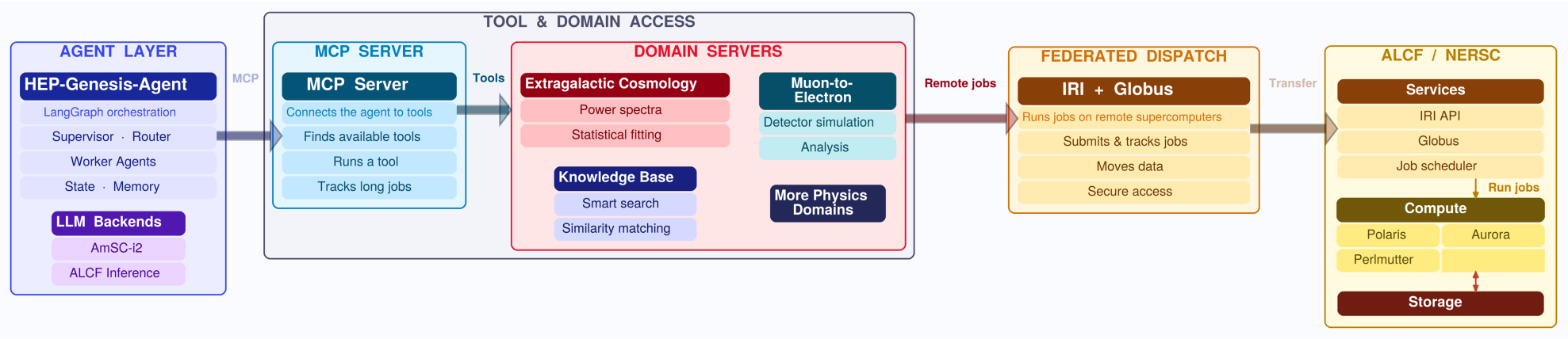
- Knowledge cut-off vs RAG/Knowledge databases
- Fabrication of synthetic data/codes vs custom tools
- Big-data handling

Custom Agentic systems vs commercial LLM harnesses

- Reasoning abilities of LLMs + coding expertise to be combined with scientific datasets
- Custom tools and custom harnesses provide a higher level of trust in the results
- Ability to work with HPC clusters and exascale codes
- Privacy and control concerns
- Scaling across large scientific user base



Agentic systems: Client + Server model



Multi-Agent systems with HPC scaling: Harnesses on local systems, hosted servers for each sub-field, APIs for control and transfer.

- Integration with Genesis Mission projects (HEP-Knowledge extraction and Quarks2Cosmos): customized agentic systems for Physics
- Tutorials/slack channels/public github repos are available outside the collaborations.
  - Significant interest from science collaborations, universities and industry partners.
- Bridge between DOE-Genesis platform and Science teams.
  - Topics: LSST-DESC data exploration, Mu2e studies, EIC simulation-based inference, DESI clustering pipelines, beamline optimization, Foundation models for HEP and NP, ….

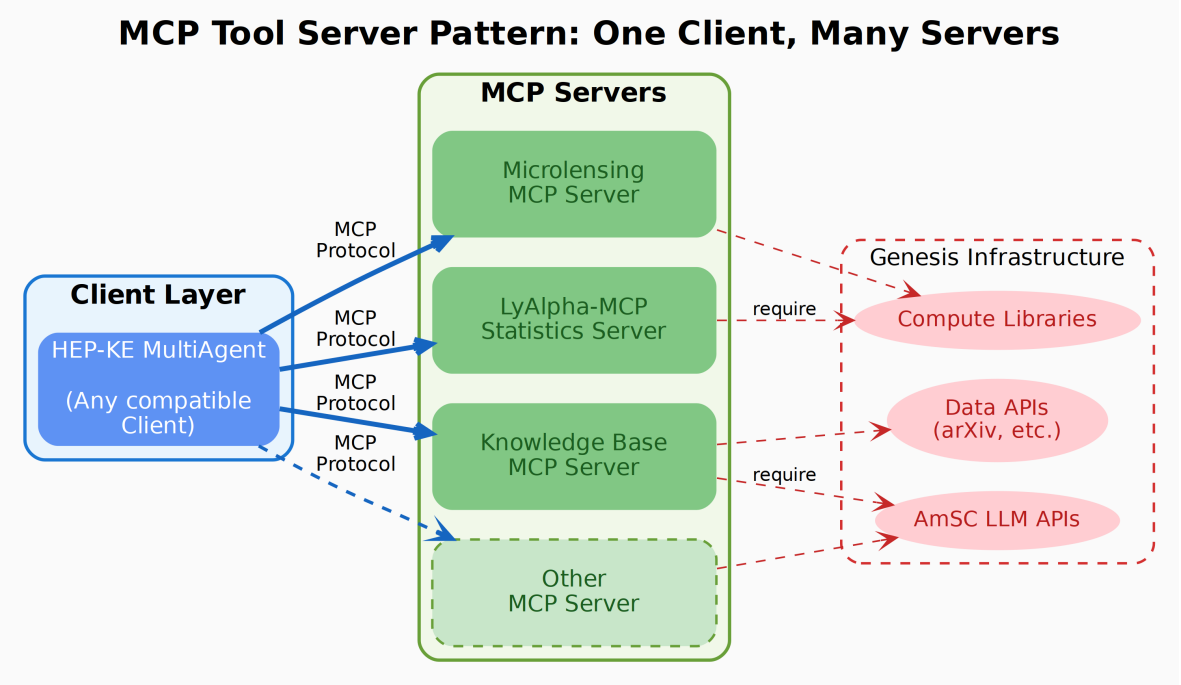
Interfacing between LLMs and tools

MCP: a standard interface between agents and tools

- Write the tool once and host on a server — any MCP-speaking client (our LangGraph client, Claude Code, CODEX) can call it.
- The server owns the science: the data files, and plotting all live server-side.
- The client only sees tool schemas and small results — fully isolated from the servers.

Multiple ways to manage MCP:

- stdio: client launches the server as a subprocess (same machine).
- streamable-http: server runs anywhere, clients connect over the network — covered today.



Today's tutorial: two python-based repos for client and server

github.com/HEP-KE/spectra-mcp-server

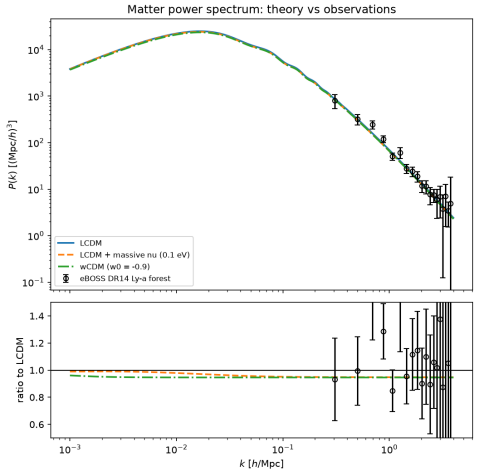
- FastMCP server exposing 4 tools around the CLASS Boltzmann code.
- Bundled eBOSS DR14 Ly-α forest P(k) data (19 bins, Chabanier+ 2019).
- notebooks/01_manual_pipeline.ipynb — the science by hand, no LLM.

github.com/HEP-KE/multiagent-client-demo

- LangGraph client, ~250 lines: two LLM roles (lead + worker).
- notebooks/02_demo_client.ipynb — the agent run, over HTTP.
- notebooks/03_next_steps.ipynb — stdio, Groq, skills, memory.

SAMPLE QUERY FOR THE AGENT (notebook 02)

```
TASK = f"""Compute the linear matter power spectrum at z=0
for three cosmologies:
    (1) standard LCDM,
    (2) LCDM with total neutrino mass 0.10 eV, and
    (3) wCDM with w0=-0.9.
Then plot all three against the eBOSS DR14 Lyman-alpha forest data,
using LCDM as the ratio reference. Save all files to {OUTPUT_DIR}."""
```



Server: tools are plain Python functions

tools/spectra_tools.py

```
@validate_call
def compute_power_spectrum(
    model: Literal["lcdm", "nu_mass", "wcdm"],
    output_dir: str,
    sum_mnu_eV: Annotated[float, Field(gt=0, le=1.0)] = 0.10,
    w0: Annotated[float, Field(ge=-2.0, le=-0.3)] = -0.9,
    ...
) -> ArtifactResult:
    """Compute a linear matter power spectrum P(k)
    with the CLASS Boltzmann code.

    Use this tool once per cosmological model. It
    writes a CSV file and returns its path. Pass the
    returned file path to plot_power_spectra —
    never copy the numbers.
    """
```

No MCP imports here

- Type hints + Field constraints + the docstring become the tool schema the agent sees.
- ArtifactResult: status, files, message, metadata.
- Arrays travel as CSV file paths — never through the LLM context.

Server: from Python package to MCP server

pyproject.toml

```
[tool.mcp-server]
tool_modules = ["tools"]
```

tools/__init__.py

```
__all__ = [
    "get_eboss_data", "list_cosmology_models",
    "compute_power_spectrum", "plot_power_spectra",
]
```

RUN IT (terminal)

```
python -m mcp_server --transport streamable-http --port 8000
# clients connect to http://127.0.0.1:8000/mcp
```

A generic wrapper

- mcp_server/ reads pyproject, imports the modules, and registers every name in __all__ as an MCP tool.
- Add your own science: drop in a module, extend __all__ — nothing else changes.

Client: connect to the server, pick an LLM

notebooks/02_demo_client.ipynb

```
HTTP_CONFIG = {
    "spectra": {
        "transport": "streamable_http",
        "url": "http://127.0.0.1:8000/mcp",
    }
}
tools = await load_tools(HTTP_CONFIG)
```

agents/llm.py

```
llm = make_llm()          # Gemini free tier (default)
llm = make_llm("groq")    # alt: open-weight gpt-oss-120b
```

MCP tools → LangChain tools

- langchain-mcp-adapters fetches the schemas; the agent/client can now call server code it has never imported.

Compatible LLM endpoints

- Free version of Gemini flash is the default. OpenAI gpt-oss-120b (open-source model hosted on groq) is an alternative.
- Any compatible LLM will work (Anthropic, OpenAI models).
- Keys live in .env, never in code.

Note:

- Gemini free tier provides 20 requests/day ≈ one full agent run
- The LLM is not state-of-the-art by any means. Token costs can rise up for paid-models.

Client: two LLM roles — lead and worker (subagents)

Lead (visited twice)

- First visit: break the task into a 2-5 step JSON plan.
- Last visit: write the final markdown report.

Worker (once per step)

- A ~10-line ReAct loop: call tools until the model replies with text.
- Fresh messages each step; file paths from earlier steps passed as context.

agents/nodes.py — the worker loop

```
for _ in range(MAX_TOOL_ITERATIONS):
    reply = await _ainvoke(bound, messages)
    messages.append(reply)
    if not reply.tool_calls:
        break
    for call in reply.tool_calls:
        result = await tools_by_name[call["name"]] \
            .ainvoke(call["args"])
        messages.append(
            ToolMessage(str(result),
                        tool_call_id=call["id"]))
```

Client: the graph — routing is plain Python

agents/graph.py

```
def route_from_worker(state):
    if state["current"] < len(state["plan"]):
        return "worker"      # more steps to run
    return "lead"            # plan done -> report

graph = StateGraph(AgentState)
graph.add_node("lead", make_lead(llm, tools))
graph.add_node("worker", make_worker(llm, tools))
graph.add_edge(START, "lead")
graph.add_conditional_edges("lead", route_from_lead,
                            ["worker", END])
graph.add_conditional_edges("worker", route_from_worker,
                            ["worker", "lead"])
return graph.compile()
```

The flow

- START → lead (plan) → worker → worker → ... → lead (report) → END

The “supervisor” is deterministic

- Two if-statements route the whole system — not every agent in a multi-agent system needs to be an LLM call.

Client: state — what flows through the graph

agents/state.py

```
class AgentState(TypedDict):
    task: str                  # the user's science question
    plan: list[Step]           # written once by the lead
    current: int               # index of the next step
    step_results: list[str]    # one summary per step
    final_report: str          # written by the lead, at the end
    history: list[str]         # previous runs (for follow-ups)
```

Agent state through a directed graph:

- Minimal setup: no worker types, no checkpointing. Production systems grow each of those.
- Every node receives the state and returns a partial update; LangGraph merges it.

What we cover today — and what we don't

Covered

- 01 — building science tools by hand: CLASS spectra vs eBOSS DR14, tools called as plain Python.
- Launch the MCP server over streamable-http; list its tools from the client.
- 02 — the agent run: plan → tool calls → report, streamed live.
- Eval: agent's figure vs the ground-truth figure, side by side.

Implemented, not covered (notebook 03)

- stdio transport, different LLM backend, skills, memory, follow-up runs

Not implemented

- Parallel workers, checkpointing / human-in-the-loop, systematic evals, auth + deployment, multiple servers.

Backup slides

Skills, Memory, Follow-ups & backends

demo in notebooks/03_next_steps.ipynb

Backup — skills: recipes loaded on demand

skills/cosmology-comparison.md

```
---
name: cosmology-comparison
description: Recipe for comparing matter power spectra
  of several cosmologies against the eBOSS data
---
# Cosmology comparison recipe
1. Call list_cosmology_models first if unsure ...
2. Compute the reference model (usually lcdm) FIRST ...
```

agents/skills.py

```
@tool
def load_skill(name: str) -> str:
    """Load the full instructions of a named skill."""
    ...
```

Skill vs tool

- A tool is a capability; a skill is know-how — HOW to use the tools well.

Progressive disclosure

- The lead only sees a name+description index at planning time; the worker loads the full text when a step needs it.
- Opt-in: build_graph(llm, tools, extras=True).

Same idea as Claude Code's Agent Skills.

Backup — memory: lessons that survive across runs

agents/memory.py

```
@tool
def remember(lesson: str) -> str:
    """Save a one-line lesson to persistent memory
    for future runs. Use this sparingly, when you
    learn something non-obvious that would help
    next time."""
    with MEMORY_FILE.open("a") as f:
        f.write(f"- {lesson.strip()}\n")
```

MEMORY.md at the repo root

- Read into the lead's planning prompt at the start of every run.
- The remember tool lets the agent append a lesson worth keeping.

The smallest possible version of the memory systems in coding agents.

- In our test runs the agent wrote its own lesson, unprompted.

Backup — follow-up runs and switching backends

agents/graph.py

```
def follow_up(previous, task):
    """A new run that remembers the previous one."""
    entry = (f"Task: {previous['task']}\n"
             f"Outcome: {previous['final_report']}")
    return {**new_run(task),
            "history": previous["history"] + [entry]}
```

SWITCHING BACKENDS

```
llm = make_llm("groq")    # that's the whole switch
```

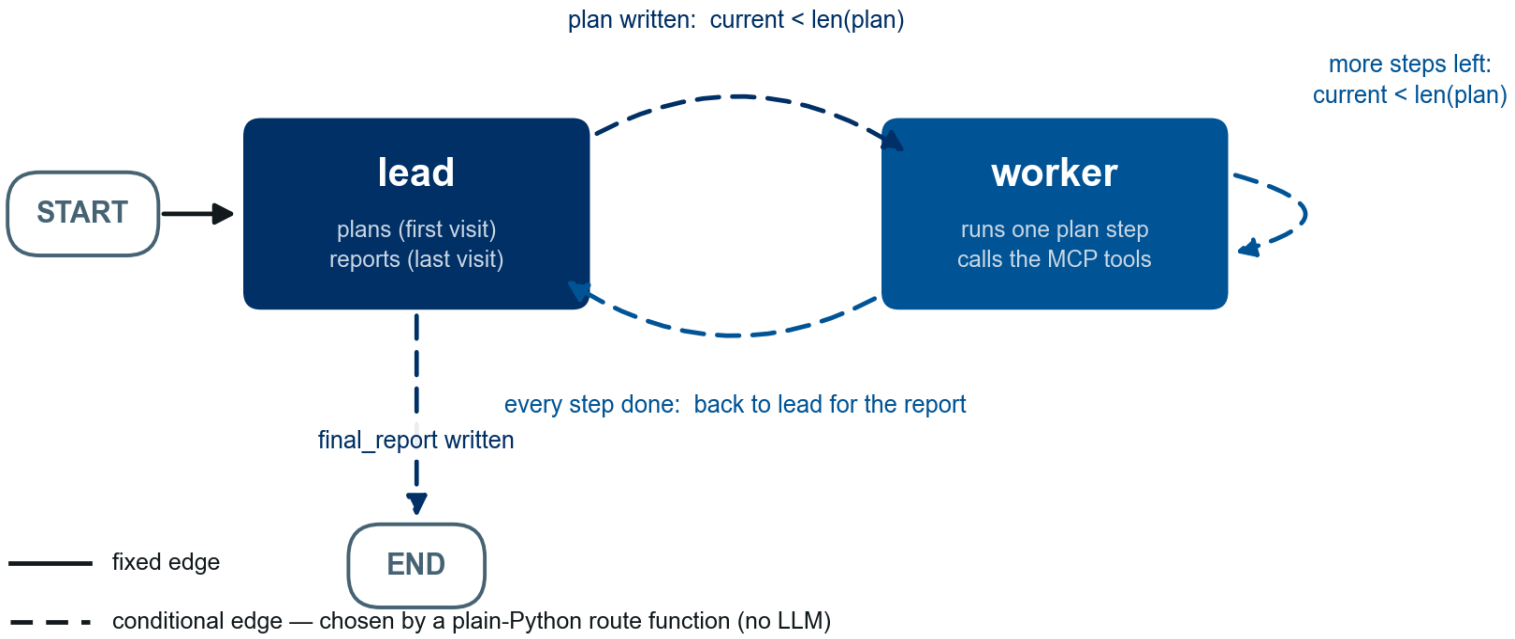
Follow-ups

- The history field carries summaries of previous runs — “now add w0=-1.1” reuses yesterday's CSVs instead of recomputing.

Backends

- No automatic fallback by design: if a key runs dry, you switch explicitly and know you did.

Backup: The graph: nodes and edges



Two LLM nodes, four edges. Both diamonds in the drawing are these two route functions.

Backup: The state mid-run: between steps 2 and 3

AgentState AFTER THE WORKER FINISHES STEP 2 (of 4)

```
{
  "task": "Compute P(k) at z=0 for three cosmologies ...",
  "plan": [
    {"id": 1, "description": "Compute the LCDM spectrum"},
    {"id": 2, "description": "Compute the 0.10 eV nu_mass spectrum"},
    {"id": 3, "description": "Compute the wCDM (w0=-0.9) spectrum"},
    {"id": 4, "description": "Plot all three vs the eBOSS data"},
  ],
  "current": 2,          # next up: plan[2] -> step 3
  "step_results": [
    "Wrote agent-output/pk_lcdm.csv",
    "Wrote agent-output/pk_nu_mass.csv",
  ],
  "final_report": "",    # still empty
  "history": [],         # first run, no follow-up
}
```

Where are we?

- The worker just finished plan[1]. route_from_worker sees current (2) < len(plan) (4) and returns “worker” — no LLM involved.

Handoffs are file paths

- step_results carries the CSV paths; the next worker visit gets them as context and reuses them instead of recomputing.
- final_report stays empty until the lead's last visit, after step 4.